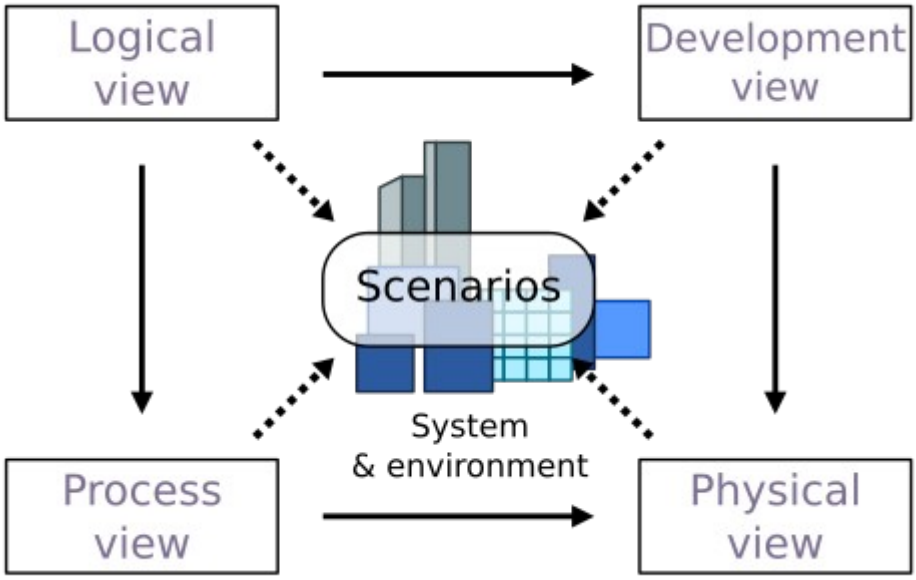
Table of Contents

1 Architectural view model

The architecture 4+1 views are used to describing the software systems based on different viewpoints, to discover if the architecture fulfills the needs of the different stakeholders. The views will be presented using the Unified Modeling Language (UML) tool



llibre

1.1 The logical view

The Logical View, often referred to as the “Class Diagram”. It focuses on the static structure of the system. This view deals with the essential components of the system, such as classes, objects, relationships, and their attributes. Class diagrams visually represent the classes and their associations in the system, to understand how various components interact with one another. This view provides insights into the overall design and organization of the system’s functionality.

Related UML diagrams:

- Class diagrams
- Component diagram

1.2 The Physical view

It concerns the topology of the software components on the physical infrastructure as well as the physical connections between components. It includes details about servers, nodes, and their interconnections, including defining the communication protocols This view ensures that the system architecture aligns with the physical constraints and requirements.

Related UML diagrams:

- Class diagrams
- State diagrams
- Component diagrams

1.3 The Development view

The system from the perspective of the programmers (or the implementation view) and is concerned with software management. It maps the logical design to physical code organization, clarifies module boundaries, dependencies, and re-usability.

1.4 The Process view

The dynamic aspects of the system, explains the system processes and how they communicate. Models the dynamic behavior, execution flow, threading, synchronization, performance, and fault tolerance.

Related UML diagrams:

- Sequence diagrams,
- Activity diagrams
- State diagrams

1.5 The Scenarios (or use case view)

It captures what the system does from an end-user perspective. The description of an architecture is illustrated using a small set of use case, or scenarios, which become a fifth view. The scenarios describe sequences of interactions between objects and between processes. They are used to identify architectural elements and to illustrate and validate the architecture design.

Related UML diagrams:

- Use Case diagram

2 Architectural guidelines

2.1 Loosely coupled architecture

The architecture should follow the guidelines for loosely coupled architecture. A loosely coupled architecture is a design principle in which system components are weakly associated. This means that each component can operate independently, allowing for greater flexibility in system design and maintenance.

Key characteristics:

- Changes made to one component have minimal impact on others, which enhances the overall resilience of the system
- Each component has little or no knowledge of the internal workings of other components.
- Systems can scale more easily as components can be modified or replaced independently.
- Each component can be developed, deployed, tested and maintained independently

2.2 Asynchronous component communication

A loosely coupled architecture using a queue allows different components of a system to communicate asynchronously, meaning they can operate independently without being directly connected.

The component communication uses the publish/subscriber model, in which a communication queue will be assigned to each of the subscribers. When a new message is published, the related message Topic will be distributed to the assigned subscribers queue.

Key characteristics:

- Independence: Producers and consumers don’t need to know about each other. They only interact with the queue, reducing direct dependencies.
- Failure isolation: If one component fails, the queue allows other components to continue functioning. This isolation minimizes the impact of failures on the overall system.
- Asynchronous Communication: Components can communicate without waiting for each other to respond, which reduces downtime and improves responsiveness.
- Buffering: They act as buffers that store messages until the receiving service is ready to process them. This helps manage varying loads and prevents service overload

3 The Deployment view

3.1 Software artifacts

3.1.1 Folder structure

Tbd

3.1.2 Github

GitHub is a cloud-based platform for hosting and managing code that allows developers to collaborate on projects using version control. It is primarily used for storing code, tracking changes, and facilitating teamwork on software development projects. In this project github will be used for:

- Version Control: to track changes in code over time.
- Repositories: to store code, enabling sharing of code and collaboration
- Pull Requests: method to propose changes to a project, allowing for code review and discussion before integration.
- Issue tracking: to track bugs and feature requests, to manage the projects workflow.

3.1.3 Music sources

tbd

3.1.4 Self-documenting python code.

Module headers should have following Google style docstring formatting

```
"""

 ####   #####     ##    #####      #     ####
#    #  #    #   #  #   #    #     #    #    #
#    #  #    #  #    #  #    #     #    #    #
#    #  #####   ######  #    #     #    #    #
#    #  #   #   #    #  #    #     #    #    #
 ####   #    #  #    #  #####      #     ####


Created on May 28, 2026
@author:        Henk Stevens & Olaf Mastenbroek & Onno Janssen
@copyright:     Copyright, Oradio Stichting
@license:       GNU General Public License (GPL)
@organization:  Oradio Stichting
@version:       1
@email:         oradioinfo@stichtingoradio.nl
@status:        Development
@summary:       One line summary of the module
@description:   Detailed description of module
"""
```

- @author: the author(s)

- @copywright, @license, @organization, @version, @email, @status: open-source related information.
- @summary: one-line summary of module
- @description: detailed description of modules

For each class, method or function there should be Google style documentation strings (docstring) , with following format:

```
def example_function(param1, param2):
    """Summary line.

    Detailed description of the function's purpose and behavior.

    Args:
        param1 (int): Description of the first parameter.
        param2 (str): Description of the second parameter.

    Returns:        bool: True if successful, False otherwise.

    Raises:        ValueError: If param1 is negative.
    Examples:
        >>> example_function(1, 'test')
        True
    """
```

- **Summary line**: A brief description of the module, class, or function, limited to one line.
- **Detailed Description:** This section provides a more comprehensive explanation of the code's purpose and usage.
- **Sections**:

Section	Description
Args	Lists the parameters accepted by the function, including their types.
Returns	Describes the return value and its type.
Raises	Lists exceptions that the function may raise, along with conditions.
Examples	Provides usage examples to illustrate how to use the function or class.

3.2 Target environment

- The RaspberryPi-3A+ headless module will be used as central control unit, providing hardware for:
  - Connectivity: WiFi, Bluetooth, Ethernet, USB
  - Input/Output pinning (GPIO): for user interface, as buttons and led indications
  - RaspiOS: Raspberry Pi OS (open-source) is a Unix-like operating system developed for the Raspberry Pi line of single-board computers. Based on Debian, a Linux distribution,

it is maintained by Raspberry Pi Holdings and optimized for the Pi's hardware, with low memory requirements and support for both 32-bit and 64-bit architectures.

- Extension interface at GPIO for Audio Playback (DigiAmp), touch buttons, volume knob and led drivers.

- Raspberry Pi OS: latest version, after being tested by developers
- Python is the programming environment, which has a wide range of libraries specifically designed for hardware interaction, created by other developers, eliminating to write everything from scratch. Comes preinstalled on the Raspi-OS

3.3 Software installation

- Development: Install script
- production: Image copy

3.4 Runtime

- runs on boot
- logging
- handling errors

3.5 Configuration

- Configuration items:
  - config.txt,
  - asound.conf,
  - mpd.conf,
  - logrotate.conf
- ??

3.6 System services

- systemd services:
  - librespot.service
  - mpd.service
  - oradio.service
  - usb_low_idle_power_service: (still needed??)
- Dynamic device management: Udev rules for USB devices
-

4 The Logical view: Class diagrams

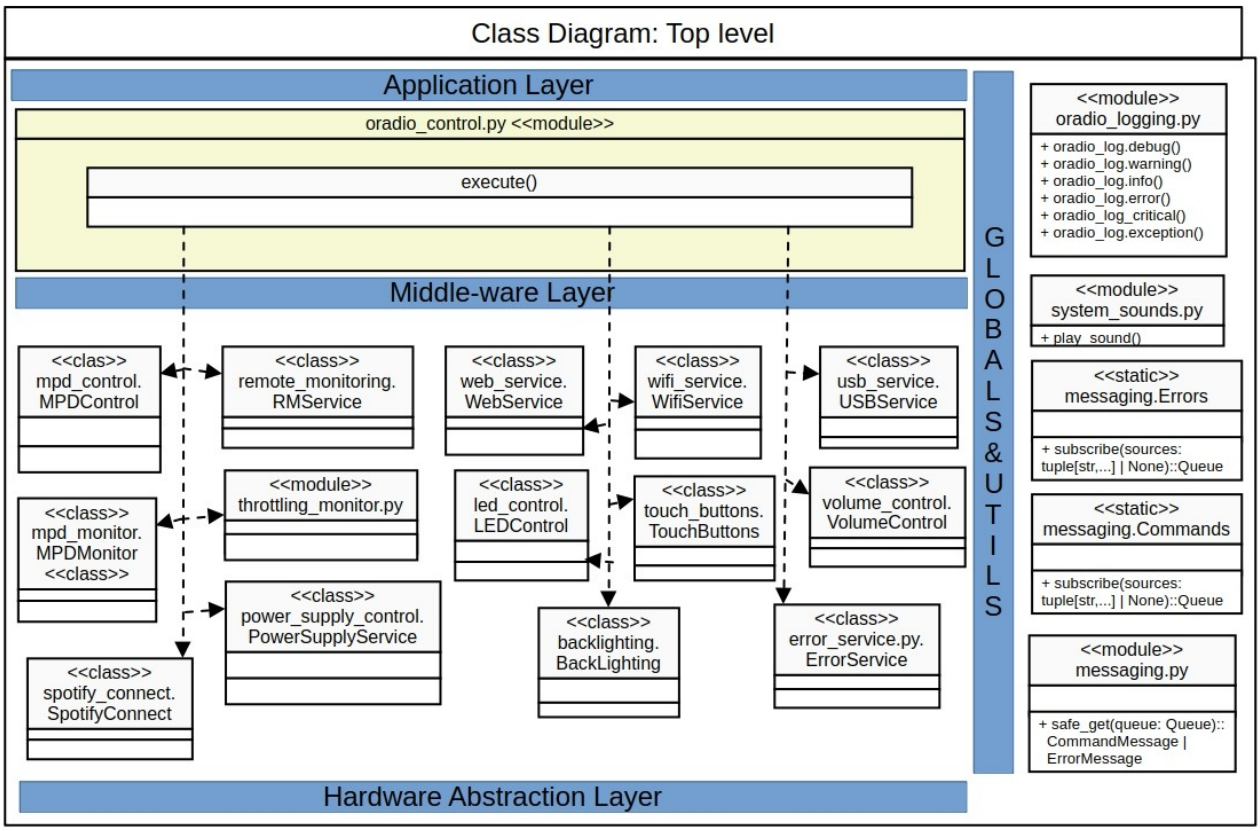
The Oradio3 software layering typically consists of three main layers:

- the **hardware abstraction layer** (HAL), which includes the physical components.
- the **middle-ware layer**, which facilitates communication and data management between the hardware and applications. The OS services is considered to be part of this layer
- the **application layer**, where the actual software applications run and perform specific tasks.
- the **globals&utils layer**, which provide global and public functions

The diagrams are organized from a top-level view and for each python module the class design.

4.1 The top level diagram

There is one application running, being the python module oradio_control.py, which will initialize and create all instances necessary for the operation. The execute() function indicates that oradio_control.py is executed and running.



Most of the classes (stereotype <<class>>) will be instantiated during initialization phase of execution.

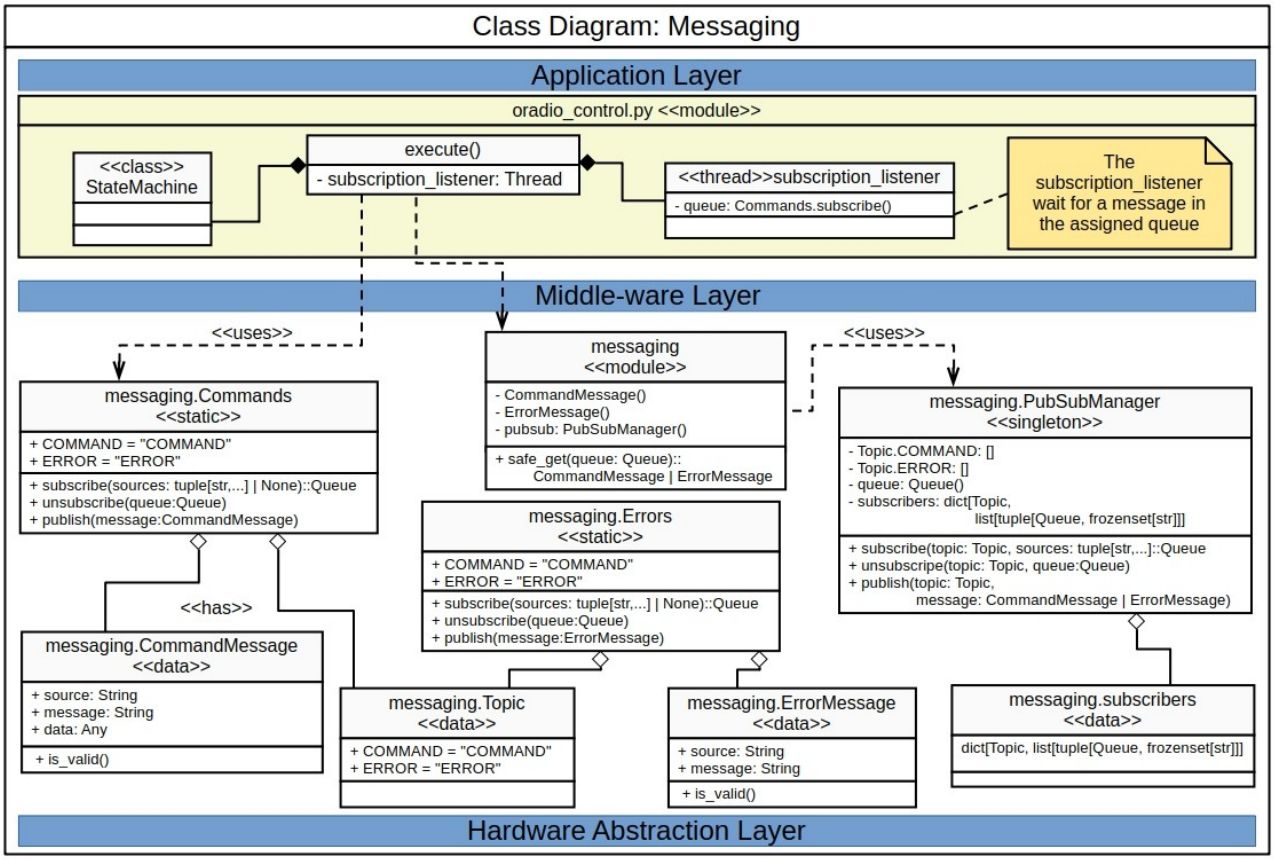
The throttled_monitor is globally instantiated, a monitoring thread will be started, to check the condition of the raspberrypi.

The global methods or functions are represented in the GLOBAL&UTILS layer, as these functions are public, being:

- oradio_log.xx()

- play_sound()
- subscribe_commands(), subscribe_errors(), publish_command(),publish_error()

4.2 Messaging



The messaging.py module provides services for the publish-subscribe model as inter-module communication. It is a messaging pattern where publishers send messages to a topic without knowing who the subscribers are, while subscribers express interest in specific topics to receive those messages. This model allows for asynchronous communication and decouples the components, enhancing scalability and flexibility in distributed systems.

The module has a public API to subscribe to a certain Topic or publish a message to a Topic. The defined Topics are:

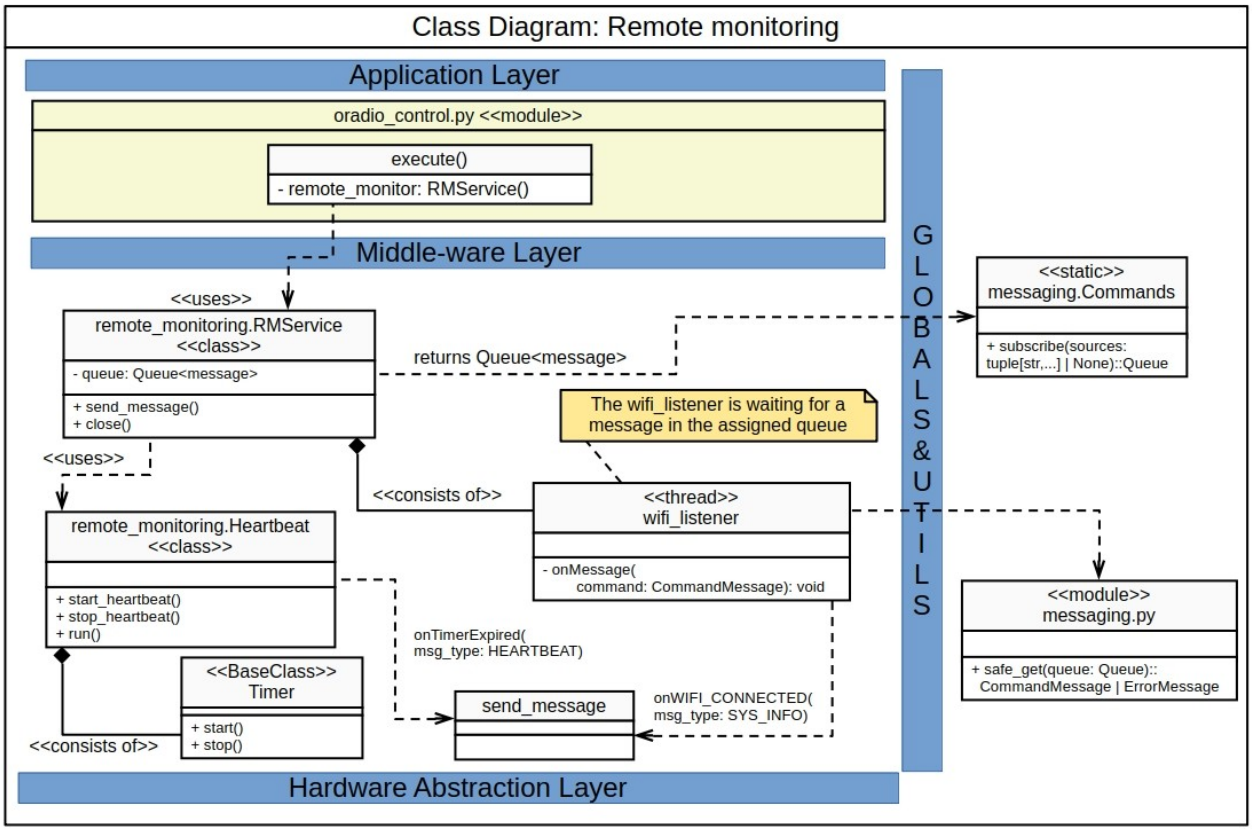
- COMMAND: representing a command
- ERROR: representing an error code

The public API provides following methods:

- subscribe_commands(): subscribe to Message Topic, returning a message queue to monitor for new commands
- publish_command(message: CommandMessage): publish a command to COMMAND Topic

- subscribe_errors(): subscribe to Error Topic, returning a message queue to monitor for new errors
- publish_error(message: ErrorMessage): publish a command to ERROR Topic

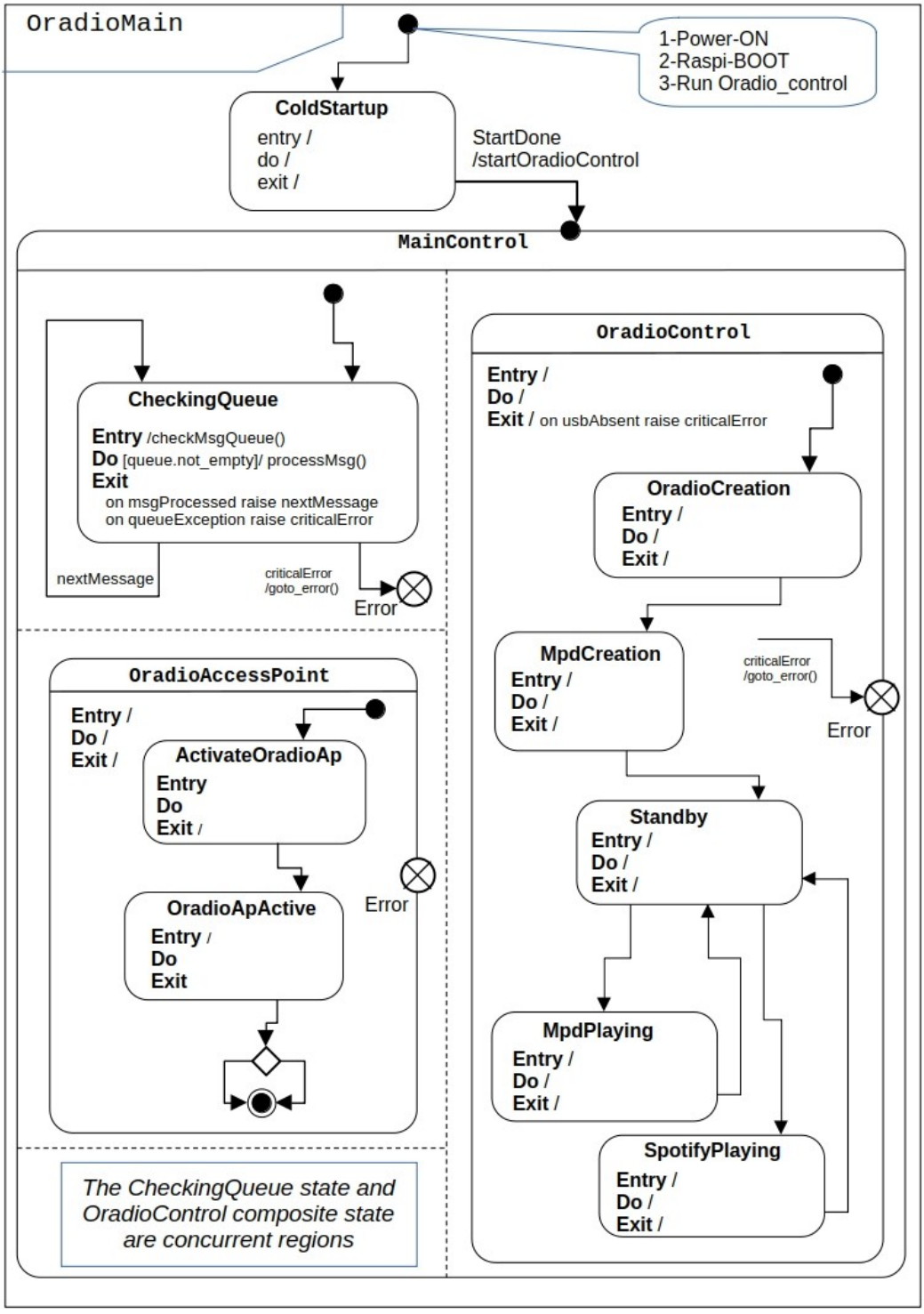
4.3 Remote Monitoring Service



The Remote Monitoring Service maintains a connection with a Remote Server via Wifi. The class
195 RMService() will be instantiated during initialization of oradio_control.py. The RMService class is depending on whether there is a WiFi connection available. So it initiates a thread <wifi_listener> that will listen to WiFi related commands, submitting a subscribe_commands(), which returns a message Queue. When WiFi is available is will send a SYS_INFO message to the server. A regular HEARTBEAT message is send to server as a keep-alive indication for the server

5 The Logical view: State diagrams

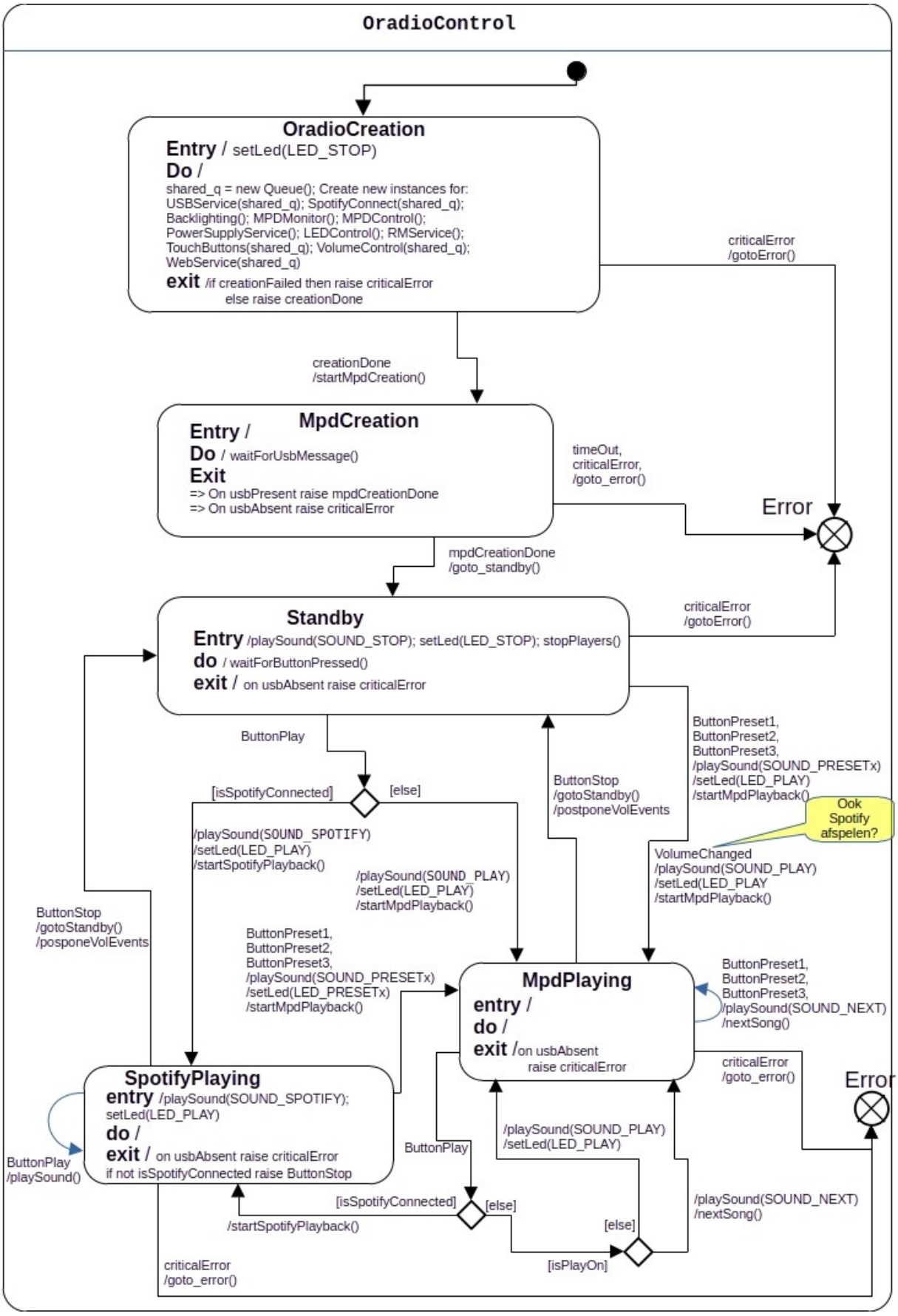
5.1 Maincontrol state



Description: xxxxxx

205

5.2 Oradio Control States

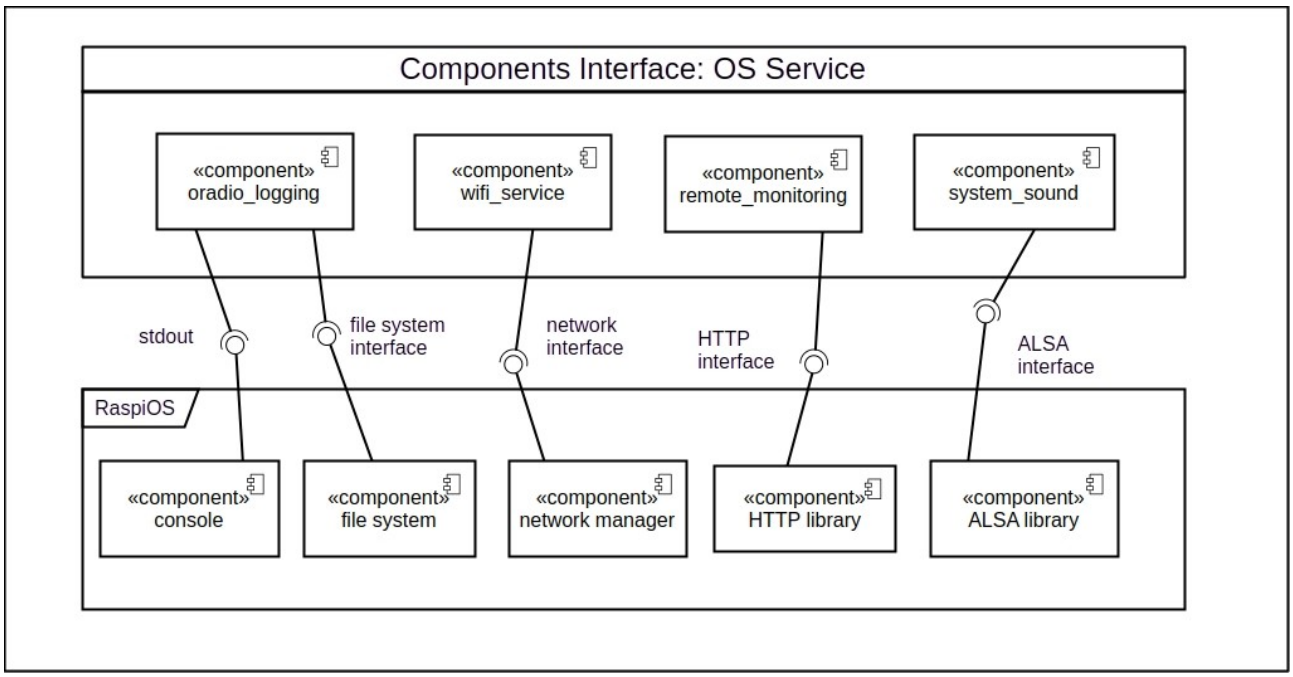


Description:xxx

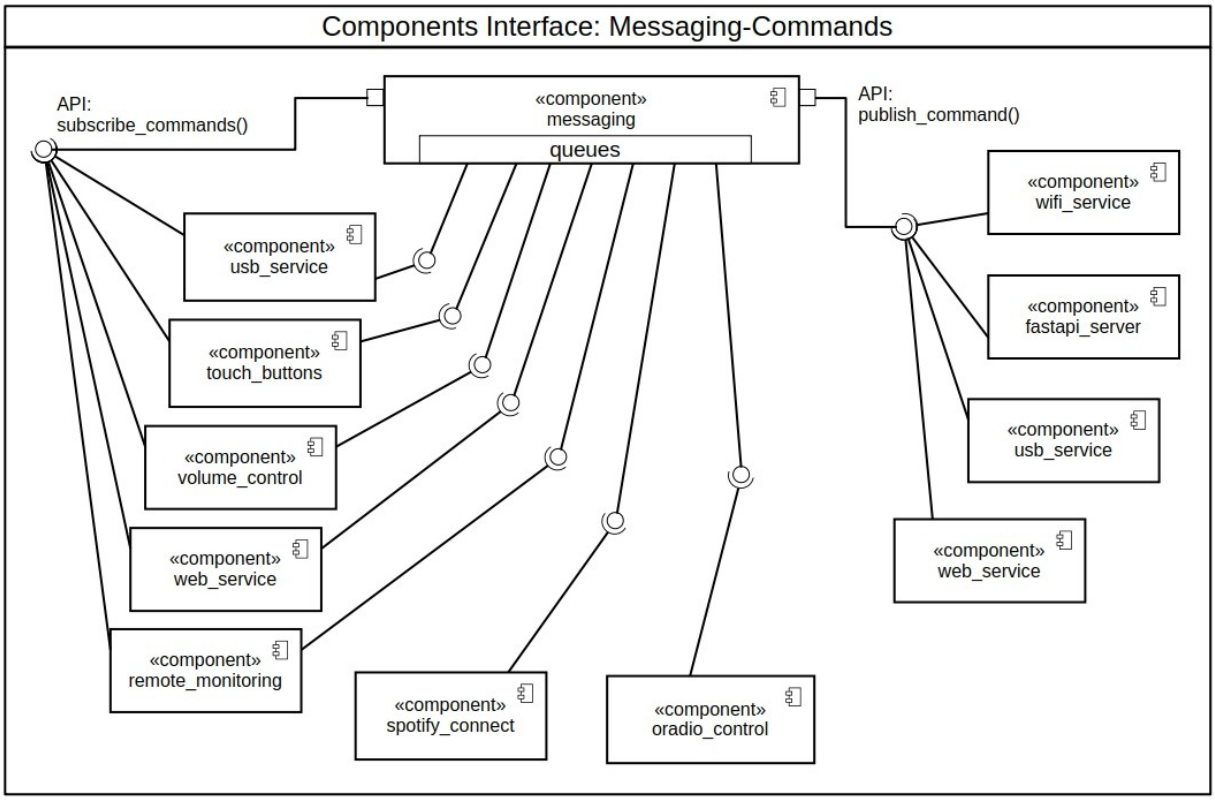
6 The Physical view: Component diagrams

6.1 OS Service

210



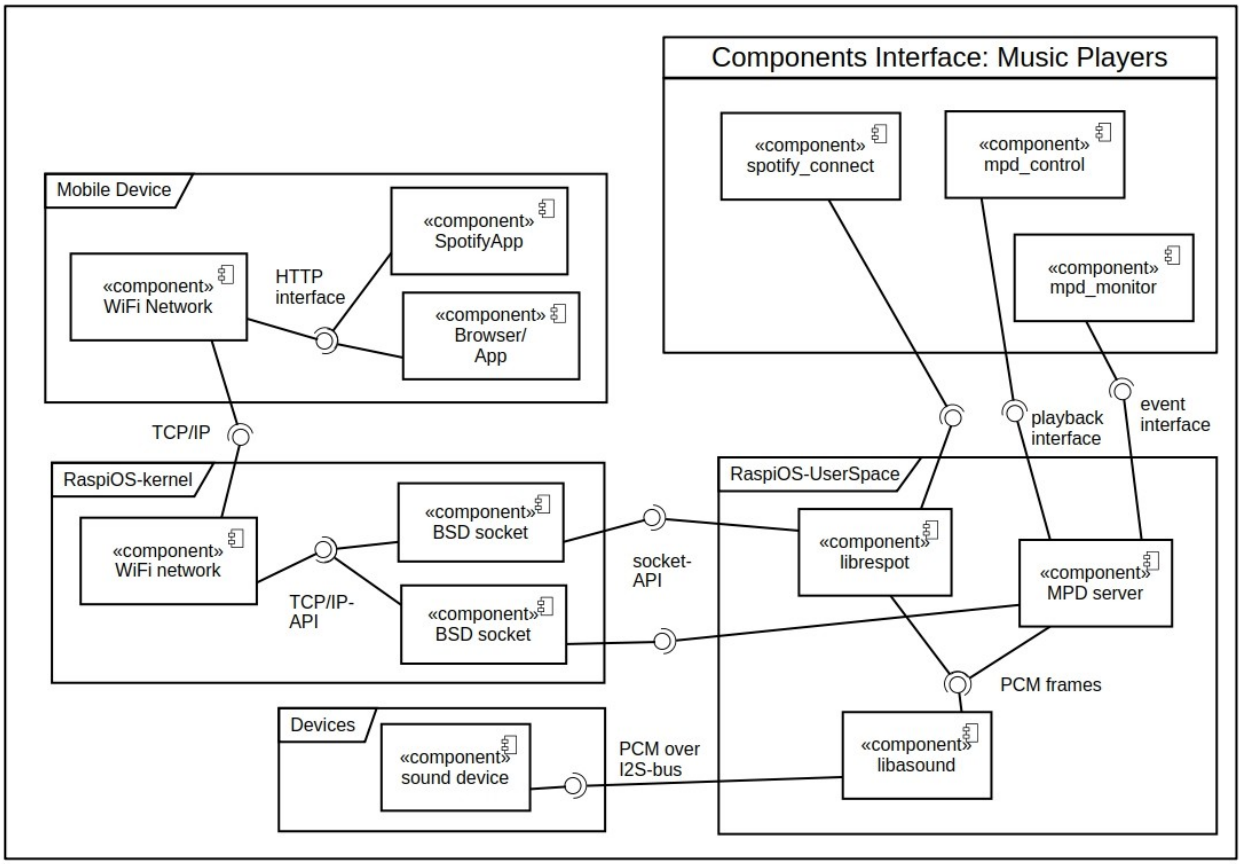
6.2 Messaging (Commands)



The messaging component interfaces to subscribers are queues. Each subscribers gets an assigned Command queue upon the provided API:subscribe_commands(). The Command queue can be monitored by the subscriber to check for new commands.

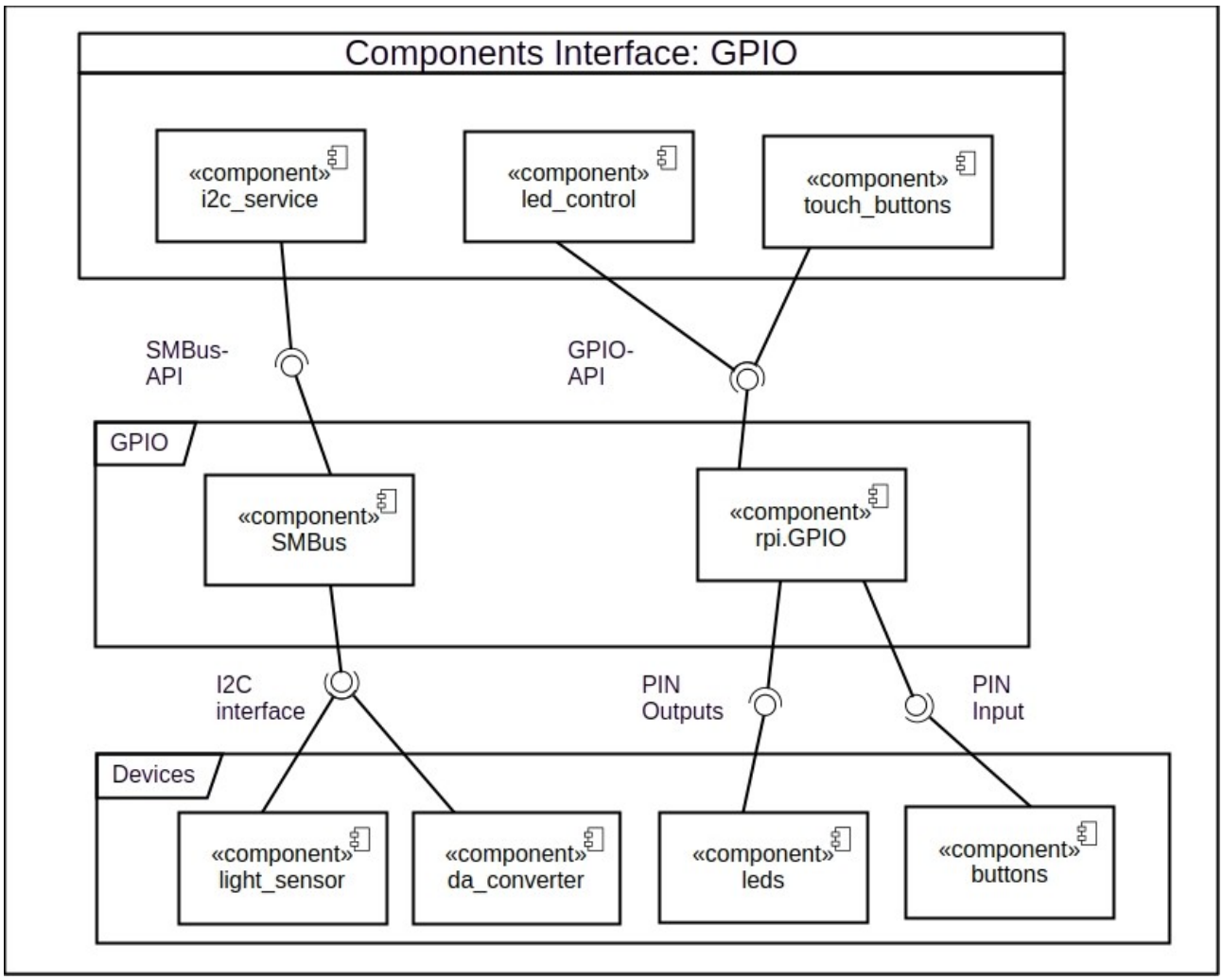
Commands can be published by publishers with the API:publish_commands(). The messaging component will distribute the published commands to all subscribers via their assigned queues.

6.3 Music Players



225

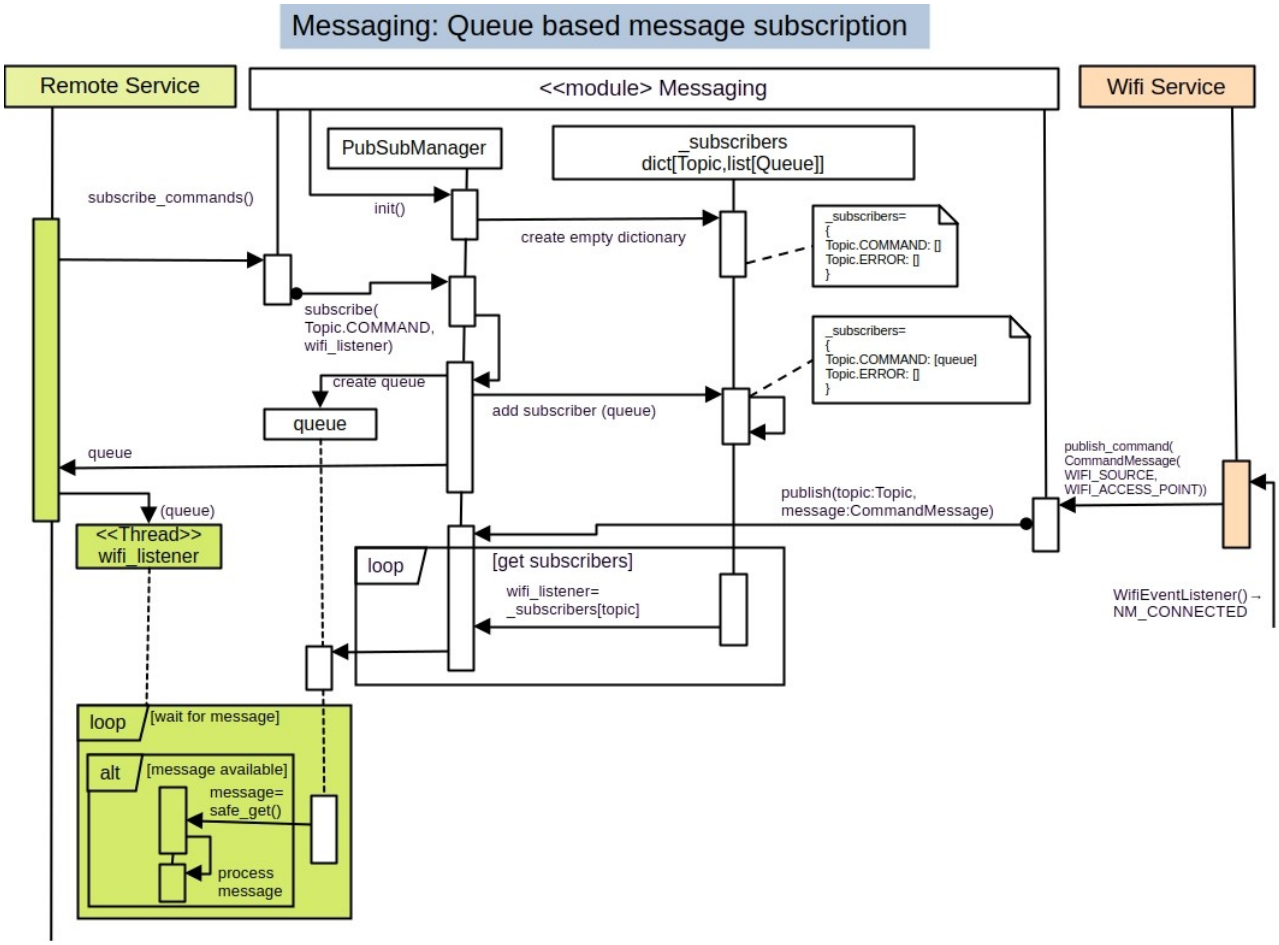
6.4 GPIO components



7 The Process view: Sequence diagrams

230

7.1 Messaging



8 The scenario view: use case diagrams